

实习生答辩

SAIE 业务域 · AI 应用工程师

员工：田沛康 **部门：**SAIE 业务域

导师：向云武 **直接主管：**李兆星

实习周期：2026.07 — 2026.08

01 自我介绍

教育与入职

02 学习及工作内容

基础与产出

03 主要工作输出及总结

两大项目

04 自我反思及下一步学习计划

收获与规划

自我介绍

教育经历

2023.09 — 2027.06 华南理工大学 计算机科学与技术

入职华为

2026.07.01 SAIE 业务域 (ICT BG) AI 应用工程师
Omni 生态 · 大数据方向

01 自我介绍

02 学习及工作内容

基础与产出

03 主要工作输出及总结

04 自我反思及下一步学习计划

学习及工作内容介绍

分类	工作学习任务	输出和收获
个人相关	大数据基础学习（Flink/Spark/鲲鹏生态）； Agent 生态调研（九问平台、27 方案）	建立流处理全貌认知；理清 skill 驱动机制
工作相关	OmniStream 表达式开发； PPT-Agentskill 开发与调优	10+ 表达式提 PR； 4 角色流水线落地； 公式原生插入 WPS 验证

学习与工作交织推进：个人相关打基础、工作相关出产出，两者相互支撑

01 自我介绍

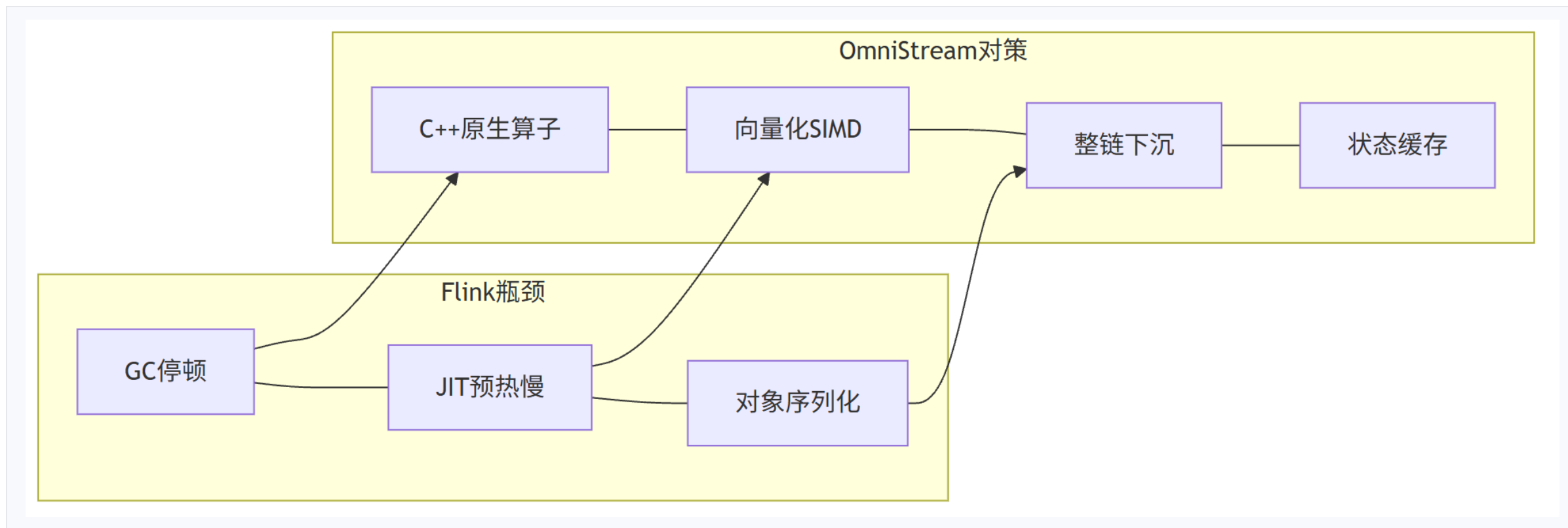
02 学习及工作内容

03 主要工作输出及总结

两大项目

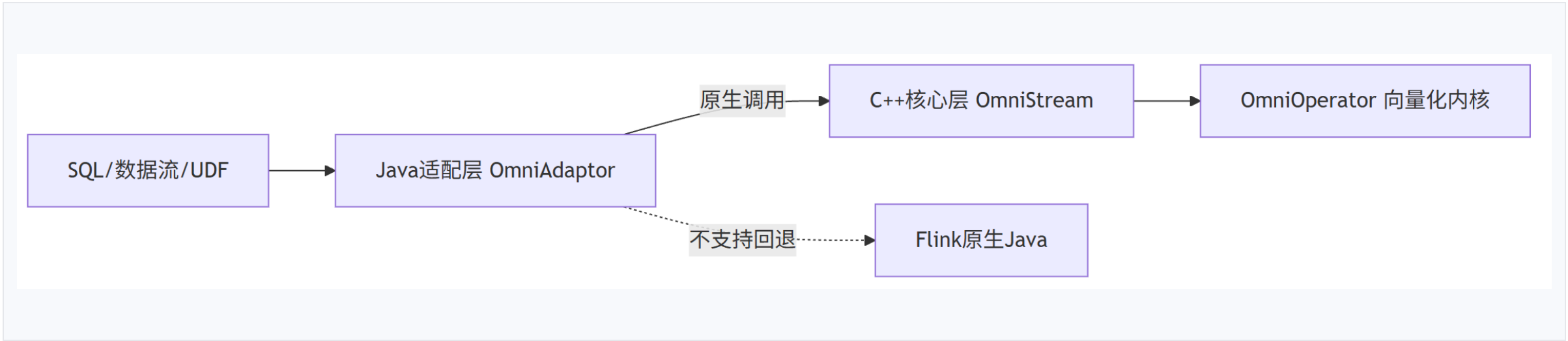
04 自我反思及下一步学习计划

项目背景：OmniStream —— 基于 Flink 生态的流处理性能加速



- Flink 跑在 JVM 上，高负载下三类瓶颈
- 内存回收停顿打断低延迟，字节码预热慢，热点才编译
- 对象序列化开销逐条放大
- OmniStream 用 C++ 重写算子，配合向量化 SIMD 指令
- 消除 JVM 开销，端到端提升流处理性能

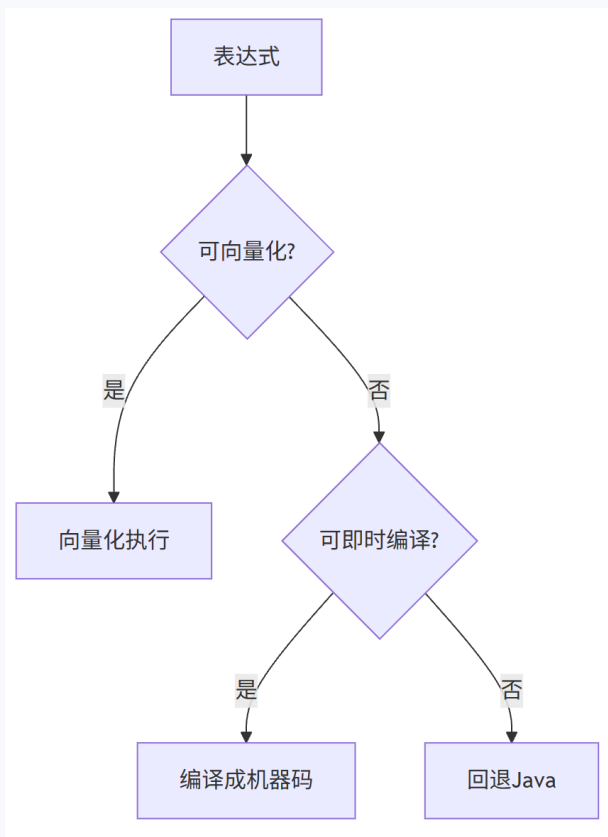
三仓库双层架构：Java 适配层 + C++ 核心层



仓库	职责	产物
OmniStream	原生运行时框架	libtnel.so
OmniAdaptor	Flink 桥接与算子替换决策	flink-tnel.jar
OmniOperator	底层向量化算子内核	5x.so + 2x.jar

零侵入接入：只改两个配置文件，不改 Flink 内核代码；不支持算子自动回退 Flink 原生 Java

表达式开发总览：让 SQL 表达式走 Native 加速路径



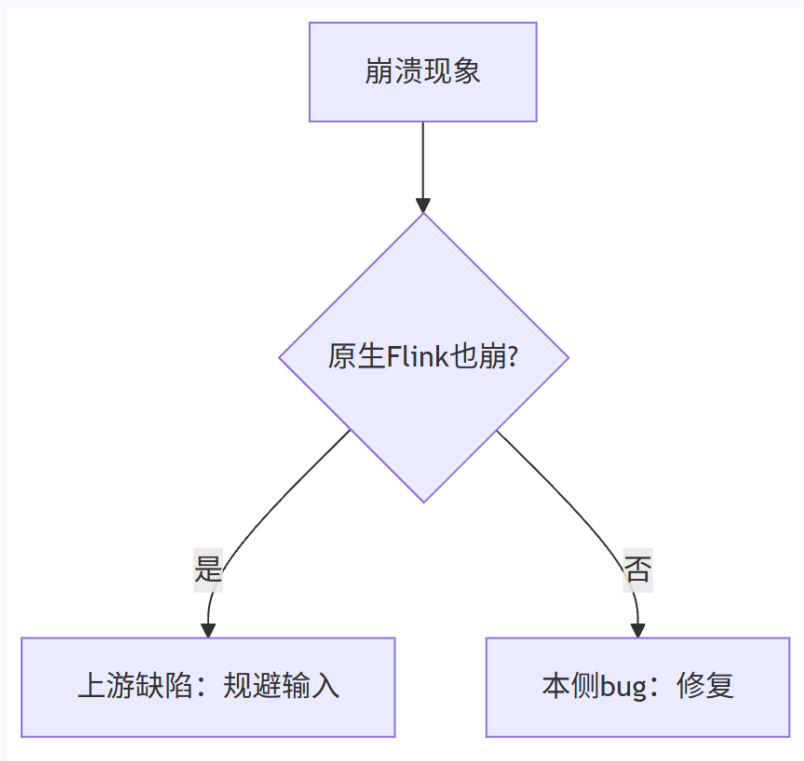
- 5 阶段生命周期：规划 → 部署 → 解析 → 编译 → 运行
- 四类分类体系：Type A 标量函数、Type B 特殊语法、Type C 聚合、Type D 别名
- 双执行后端：向量化优先，即时编译次选，都不行回退 Java
- 选路公式：优先向量化，不行才即时编译，都不行回退 Java

表达式开发案例：三类范式覆盖

案例	范式	要点
IFNULL	别名映射	语义等价已有函数，一行映射完成原生化
LEFT/RIGHT	纯向量化	按 UTF-8 码点切片，绝不切断多字节字符
BETWEEN	借原语编译	语义可拆成两个比较，借已有原语组合执行
SIMILAR TO	专用函数	正则不可拆，需专用函数解释执行

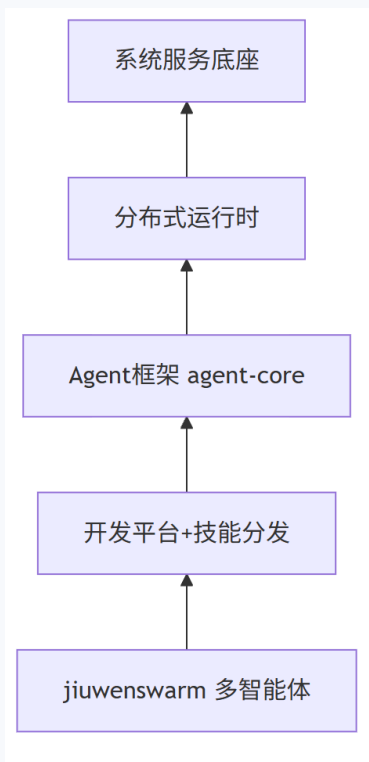
一映射、一向量化、一组合、一专用：四类范式覆盖表达式开发全场景

问题排查：BETWEEN 崩溃定位与 vanilla 对照组二分法



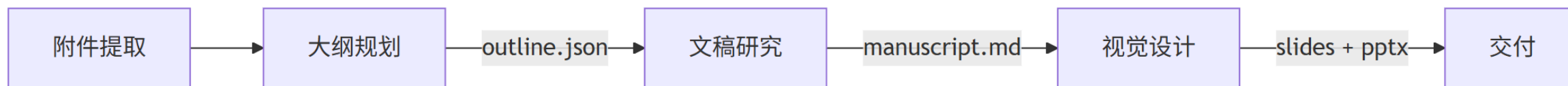
- 问题：BETWEEN 在反向区间下崩溃
- 方法：用原生 Flink 做对照组二分法
- 投影路径：原生也崩 = Flink 源码缺陷，规避输入
- 过滤路径：原生正常 = 本侧 bug，修复
- 让“甩锅还是背锅”有客观依据

项目背景：AgentOS —— 一体机办公 Agent 与 PPT-Agentskill



- 基于九问 Agent 平台开发一体机办公 Agent
- 把大模型能力做成本地化、可编辑交付的办公智能体
- 产物必须可编辑（中文办公刚需）
- 本地化部署，接一体机模型降本
- WPS 兼容性是硬约束，催生公式原生插入核心难题

4 角色流水线架构: attachment-reader → planner → researcher → designer



- 4 角色各司其职，强制分工与校验
- attachment-reader: MinerU-first 提取结构化内容
- ppt-planner: 设计大纲结构 + 需求对齐
- ppt-researcher: 调研信息 + 写文稿 + 自审
- ppt-designer: 视觉平衡 HTML 幻灯片 + 逐页 QA
- 双层质量保证: 脚本硬校验 + LLM 自审

PPTv2agent 参考：DeepPresenter 双 Agent 架构与 Content Style



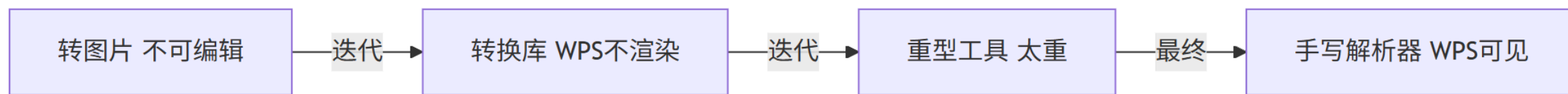
- DeepPresenter ACL2026 SOTA (均分 4.44 超 Gamma 4.36)
- 双 Agent 共享观察空间 + 环境接地反思
- 信息加工成半成品而非原材料
- 每页围绕一个核心洞察，金字塔原则
- 图片承载信息而非填空，优先可信来源
- 学术参考方法论，PPT-Agentskill 是工程落地

skill 框架比较： ppt-pipeline-swarm vs 标准 swarm-skill vs PPTv2agent

维度	标准 swarm-skill	ppt-pipeline-swarm
编排	工具跑原语	手动建队编排
质量门	脚本内嵌	脚本 + LLM 双层
人机交互	原语	转达协议

- 相比标准形态：简化为手动编排，适合固定流水线
- 相比 PPTv2agent：平台上的工程实现，非学术方法
- 设计器引擎源自 PPTAgent v2 提取重写为独立 CLI
- 选型依据：固定流水线 + 断点续跑 → 编排式

项目背景：公式原生插入 —— 可编辑 OMMML 方程而非图片



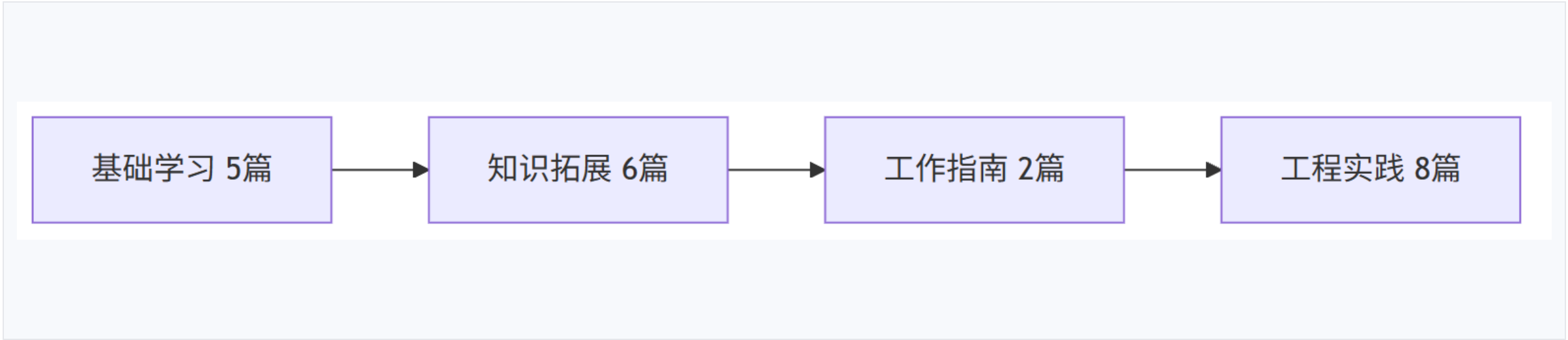
- 需求：公式必须可编辑，WPS 兼容
- Python PPT 库不支持原生公式（2019 至今未解决）
- PowerPoint 需特殊命名空间包装
- WPS 渲染有多个兼容陷阱：不渲染直立体、不认某些包装
- 方案四次迭代：图片 → 转换库 → 重型工具 → 手写解析器

产出：公式原生插入方案与验证



- 手写递归下降解析器直接生成 OMML
- 全链路：文稿 → 设计器标记 → 收集 → 注入原生方程
- 26 个公式全部成功，WPS 可见
- 零新增依赖，复用已有库
- 文件更小（32KB vs 图片 96KB）
- 配套公式识别规范决策表与校验脚本

Wiki 产出：工作指南、知识拓展、基础学习、工程实践 21 篇



分类	篇数	内容
基础学习	5	PPTAgent 框架解析、九问 skill 机制、swarmflow 原语
知识拓展	6	4 种组装方式选型、公式插入 8 项目对比、样式模板调研
工作指南	2	PresentBench 30 条检查项、三 agent 产物设计原则
工程实践	8	4 分支审计 + 纯净性总结，50+ commit 可追溯

- 21 篇约 34 万字，5 个开源项目横向调研
- 公式方案 26 公式端到端验证，12 套 CSS 模板确定性物化

01 自我介绍

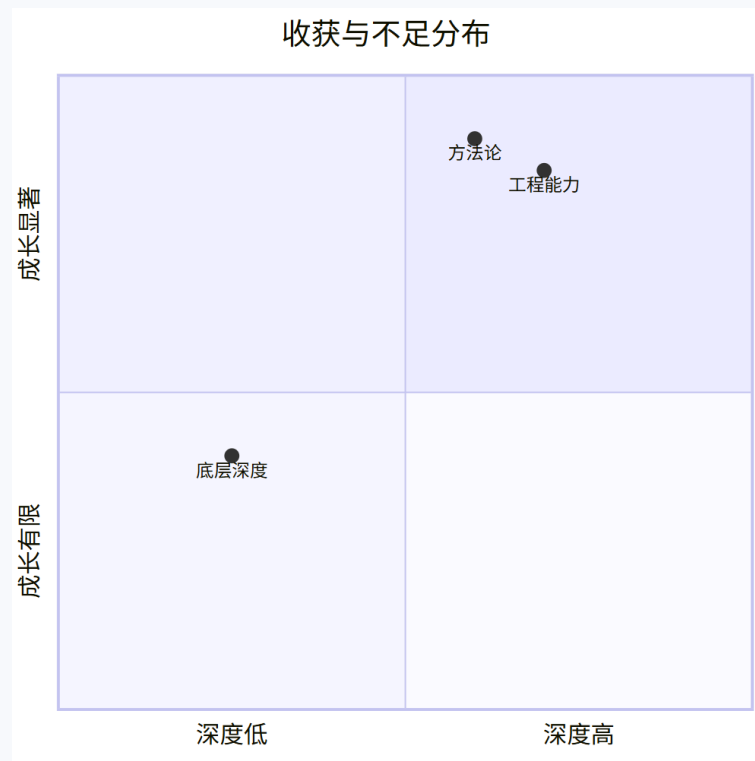
02 学习及工作内容

03 主要工作输出及总结

04 自我反思及下一步学习计划

收获与规划

收获和体会 / 有待改进之处



- 收获：代码开发能力提高、培养工程化思维
- 收获：方法论沉淀（vanilla 二分法、可观察性、纯净原则）
- 不足：底层向量化内核与编译器实现了解不够
- 不足：skill 包零实战验证，非工作流提炼
- 不足：8 月切入 Agent 后 OmniStream 深度推进受限

OmniStream开发推进

底层深度补齐

Agent工程化实战

- OmniStream 开发推进：补齐剩余表达式类型，推进性能验证与开源贡献
- 底层深度补齐：深入向量化内核与编译器实现，从能用走向吃透
- Agent 工程化实战：推进 skill 包真实场景跑通，深化公式原生插入能力扩展

致 谢

感谢在座的评委、各位领导百忙中抽出时间参与本次答辩！

感谢导师向云武、主管李兆星、周围的同事的指导与帮助！

感谢所有给予过帮助指导和工作支持的人们！